

Practical-week-2

Task-1

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user Space and kernel space during execution.

```
GNU nano 7.2 filecopy.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source_fd, destination_fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    // Check command-line arguments
    if (argc != 3)
    {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    // Open source file for reading
    source_fd = open(argv[1], O_RDONLY);

    if (source_fd == -1)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open destination file for writing
    destination_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination_fd == -1)
    {
        perror("Error opening destination file");
        close(source_fd);
        return 1;
    }

    // Read from source and write to destination
    while ((bytes_read = read(source_fd, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(destination_fd, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("Error writing to destination file");
            close(source_fd);
            close(destination_fd);
            return 1;
        }
    }
}
```

```

root@Shamikahasini:~# nano filecopy.c
root@Shamikahasini:~# gcc filecopy.c -o filecopy
root@Shamikahasini:~# echo "Hello, this is a Linux system call experiment." > source.txt
root@Shamikahasini:~# cat source.txt
Hello, this is a Linux system call experiment.
root@Shamikahasini:~# ./filecopy source.txt destination.txt
File copied successfully.
root@Shamikahasini:~# cat destination.txt
Hello, this is a Linux system call experiment.
root@Shamikahasini:~# source_fd = open(argv[1], O_RDONLY);
-bash: syntax error near unexpected token '('
root@Shamikahasini:~# ls -l
total 264
-rw-r--r-- 1 root root 80 Jul 28 06:51 FIRSST.c
-rw-r--r-- 1 root root 81 Jul 28 07:06 FIRSST.c.save
drwxr-xr-x 11 root root 4096 Aug 5 03:57 OSSP
drwxr-xr-x 4 root root 4096 Aug 7 09:10 OSSP-2
drwxr-xr-x 4 root root 4096 Aug 1 03:52 OSSP1
-rwxr-xr-x 1 root root 16344 Aug 18 07:01 command_resolver
-rw-r--r-- 1 root root 1133 Aug 18 07:01 command_resolver.c
-rw-r--r-- 1 root root 47 Aug 25 15:18 destination.txt
-rwxr-xr-x 1 root root 16264 Aug 25 15:18 filecopy
-rw-r--r-- 1 root root 1486 Aug 25 15:18 filecopy.c
-rwxr-xr-x 1 root root 16256 Aug 8 02:58 fork_demo
-rw-r--r-- 1 root root 1032 Aug 8 02:58 fork_demo.c
-rwxr-xr-x 1 root root 16328 Aug 11 06:31 parser
-rw-r--r-- 1 root root 1336 Aug 11 06:41 parser.c
-rwxr-xr-x 1 root root 16528 Aug 25 15:03 process
-rw-r--r-- 1 root root 1460 Aug 25 15:03 process.c

```

Observation: The program successfully copied the contents of source.txt to destination.txt using the open(), read(), write(), and close() system calls. The contents of both files were verified to be identical. During execution, the user-space program requested file operations from the Linux kernel through system calls, and the kernel performed the required file operations before returning control to the user-space program

Task-2

Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system 3 calls and identify the kernel services involved

```

root@Shamikahasini:~# cat sample.txt
cat: sample.txt: No such file or directory
root@Shamikahasini:~# echo "Hello Linux System Calls" > sample.txt
root@Shamikahasini:~# cat sample.txt
Hello Linux System Calls
root@Shamikahasini:~# strace --version
Command 'strace' not found, but can be installed with:
apt install strace
root@Shamikahasini:~# sudo apt update
sudo apt install strace
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1229 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [969 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [286 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1687 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [206 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1202 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [337 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1486 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [339 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [12.8 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:19 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5760 B]

```

Observation:

The strace utility successfully traced the system calls generated by the cat sample.txt command. The trace showed openat() being used to open sample.txt, followed by file and terminal operations using splice(), pipe(), and fcntl(). The close() system call was used to close the file descriptors, and exit_group(0) indicated successful program termination. The observed system calls demonstrate how the cat command interacts with Linux kernel services to access and display file contents.